Exercise 1: Build the `CSVReader` Class

The `CSVReader` class encapsulates loading a CSV file and previewing its first rows.

In [20]:
```python
from pathlib import Path
from abc import ABC, abstractmethod
import pandas as pd


class CSVReader:
    def __init__(self, file_path: str | Path):
        self.file_path = Path(file_path)
        if not self.file_path.is_file():
            raise FileNotFoundError(f"CSV file not found: {self.file_path}")
        self.data = pd.read_csv(self.file_path)

    def preview(self, n: int = 5) -> pd.DataFrame:
        if n < 0:
            raise ValueError("n must be greater than or equal to 0")
        preview_data = self.data.head(n)
        print(preview_data)
        return preview_data


reader = CSVReader('sample_data (2).csv')
print(f"Loaded file: {reader.file_path}")
print(f"Rows: {len(reader.data)}, Columns: {len(reader.data.columns)}")
reader.preview(5)
```

```
Loaded file: sample_data (2).csv
Rows: 7, Columns: 7
   id     name   age  height_cm  weight_kg         city  score
0   1    Alice  29.0      165.0       68.0     New York   85.0
1   2      Bob   NaN      172.0        NaN  Los Angeles   90.0
2   3  Charlie  35.0      168.0       72.0      Chicago    NaN
3   4    David   NaN        NaN       80.0      Houston   75.0
4   5      Eva  27.0      160.0       55.0     New York   88.0
```

Out[20]:

	id	name	age	height_cm	weight_kg	city	score
0	1	Alice	29.0	165.0	68.0	New York	85.0
1	2	Bob	NaN	172.0	NaN	Los Angeles	90.0
2	3	Charlie	35.0	168.0	72.0	Chicago	NaN
3	4	David	NaN	NaN	80.0	Houston	75.0
4	5	Eva	27.0	160.0	55.0	New York	88.0

Exercise 2: Strategy Pattern for Missing Values

A `DataCleaner` switches between interchangeable missing-value strategies without changing its main code.

In [21]:
```python
class MissingValueStrategy(ABC):
    @abstractmethod
    def handle(self, df: pd.DataFrame) -> pd.DataFrame:
        """Return a cleaned copy of the DataFrame."""
        raise NotImplementedError


class DropMissing(MissingValueStrategy):
    def handle(self, df: pd.DataFrame) -> pd.DataFrame:
        return df.dropna().reset_index(drop=True)


class FillMean(MissingValueStrategy):
    def handle(self, df: pd.DataFrame) -> pd.DataFrame:
        result = df.copy()
        numeric_columns = result.select_dtypes(include='number').columns
        result[numeric_columns] = result[numeric_columns].fillna(
            result[numeric_columns].mean()
        )
        return result


class FillMode(MissingValueStrategy):
    def handle(self, df: pd.DataFrame) -> pd.DataFrame:
        result = df.copy()
        for column in result.columns:
            modes = result[column].mode(dropna=True)
            if not modes.empty:
                result[column] = result[column].fillna(modes.iloc[0])
        return result


class DataCleaner:
    def __init__(self, strategy: MissingValueStrategy):
        self.strategy = strategy

    def set_strategy(self, strategy: MissingValueStrategy) -> None:
        self.strategy = strategy

    def clean(self, df: pd.DataFrame) -> pd.DataFrame:
        return self.strategy.handle(df)
```

In [22]:
```python
cleaner = DataCleaner(DropMissing())
drop_result = cleaner.clean(reader.data)
print('DropMissing result:')
display(drop_result)
```

```
cleaner.set_strategy(FillMean())
mean_result = cleaner.clean(reader.data)
print('FillMean result:')
display(mean_result)

cleaner.set_strategy(FillMode())
mode_result = cleaner.clean(reader.data)
print('FillMode result:')
display(mode_result)

assert drop_result.isna().sum().sum() == 0
assert mean_result[['age', 'height_cm', 'weight_kg', 'score']].isna().sum().
assert mode_result.isna().sum().sum() == 0
print('All missing-value strategies completed successfully.')
```

```
DropMissing result:
```

	id	name	age	height_cm	weight_kg	city	score
0	1	Alice	29.0	165.0	68.0	New York	85.0
1	5	Eva	27.0	160.0	55.0	New York	88.0

```
FillMean result:
```

	id	name	age	height_cm	weight_kg	city	score
0	1	Alice	29.00	165.0	68.0	New York	85.0
1	2	Bob	30.25	172.0	70.0	Los Angeles	90.0
2	3	Charlie	35.00	168.0	72.0	Chicago	86.0
3	4	David	30.25	167.0	80.0	Houston	75.0
4	5	Eva	27.00	160.0	55.0	New York	88.0
5	6	Frank	30.00	175.0	85.0	Los Angeles	86.0
6	7	Grace	30.25	162.0	60.0	Chicago	92.0

```
FillMode result:
```

	id	name	age	height_cm	weight_kg	city	score
0	1	Alice	29.0	165.0	68.0	New York	85.0
1	2	Bob	27.0	172.0	55.0	Los Angeles	90.0
2	3	Charlie	35.0	168.0	72.0	Chicago	75.0
3	4	David	27.0	160.0	80.0	Houston	75.0
4	5	Eva	27.0	160.0	55.0	New York	88.0
5	6	Frank	30.0	175.0	85.0	Los Angeles	75.0
6	7	Grace	27.0	162.0	60.0	Chicago	92.0

```
All missing-value strategies completed successfully.
```

Exercise 3: Decorators for Logging and Timing

Decorators add reusable logging and timing behavior without changing the core methods.

In [23]:
```
from functools import wraps
from time import perf_counter

def log_action(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        print(f"Starting {func.__name__}...")
        result = func(*args, **kwargs)
        print(f"Finished {func.__name__}.")
        return result
    return wrapper

def log_time(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        start = perf_counter()
        result = func(*args, **kwargs)
        print(f"{func.__name__} took {perf_counter() - start:.6f} seconds.")
        return result
    return wrapper

CSVReader.preview = log_action(log_time(CSVReader.preview))
DataCleaner.clean = log_action(log_time(DataCleaner.clean))
```

In [24]:
```
decorated_reader = CSVReader('sample_data (2).csv')
print('Decorated CSVReader.preview:')
decorated_reader.preview(3)

decorated_cleaner = DataCleaner(FillMean())
print('\nDecorated DataCleaner.clean:')
cleaned_data = decorated_cleaner.clean(decorated_reader.data)
display(cleaned_data)

assert decorated_reader.preview.__name__ == 'preview'
assert decorated_cleaner.clean.__name__ == 'clean'
assert cleaned_data.isna().sum().sum() == 0
print('Logging and timing decorators completed successfully.')
```

```
Decorated CSVReader.preview:
Starting preview...
   id     name   age  height_cm  weight_kg         city  score
0   1    Alice  29.0      165.0       68.0     New York   85.0
1   2      Bob   NaN      172.0        NaN  Los Angeles   90.0
2   3  Charlie  35.0      168.0       72.0      Chicago    NaN
preview took 0.001114 seconds.
Finished preview.

Decorated DataCleaner.clean:
Starting clean...
clean took 0.002308 seconds.
Finished clean.
```

	id	name	age	height_cm	weight_kg	city	score
0	1	Alice	29.00	165.0	68.0	New York	85.0
1	2	Bob	30.25	172.0	70.0	Los Angeles	90.0
2	3	Charlie	35.00	168.0	72.0	Chicago	86.0
3	4	David	30.25	167.0	80.0	Houston	75.0
4	5	Eva	27.00	160.0	55.0	New York	88.0
5	6	Frank	30.00	175.0	85.0	Los Angeles	86.0
6	7	Grace	30.25	162.0	60.0	Chicago	92.0

```
Logging and timing decorators completed successfully.
```

Exercise 4: Factory Pattern for Data Transformations

The factory creates transformation objects from names, while every transformation follows the same `DataTransform` interface.

In [25]:
```python
import pandas as pd
from abc import ABC, abstractmethod
```

In [26]:
```python
class DataTransform(ABC):
    @abstractmethod
    def apply(self, df: pd.DataFrame) -> pd.DataFrame:
        """Return a transformed copy of the input DataFrame."""
        raise NotImplementedError


class NormalizeColumns(DataTransform):
    """Normalize numeric columns to the range [0, 1]."""

    def apply(self, df: pd.DataFrame) -> pd.DataFrame:
        result = df.copy()
        for column in result.select_dtypes(include='number').columns:
            minimum = result[column].min()
```

```
            maximum = result[column].max()
            if pd.notna(minimum) and pd.notna(maximum) and maximum != minimu
                result[column] = (result[column] - minimum) / (maximum - mir
            elif pd.notna(minimum):
                result[column] = 0.0
        return result


class RemoveDuplicates(DataTransform):
    def apply(self, df: pd.DataFrame) -> pd.DataFrame:
        return df.drop_duplicates().reset_index(drop=True)


class StandardizeText(DataTransform):
    def apply(self, df: pd.DataFrame) -> pd.DataFrame:
        result = df.copy()
        for column in result.select_dtypes(include=['object', 'string']).col
            result[column] = result[column].map(
                lambda value: value.strip().lower() if isinstance(value, str
            )
        return result


class TransformFactory:
    _TRANSFORMS = {
        'normalize': NormalizeColumns,
        'normalize_columns': NormalizeColumns,
        'remove_duplicates': RemoveDuplicates,
        'duplicates': RemoveDuplicates,
        'standardize_text': StandardizeText,
        'text': StandardizeText,
    }

    @classmethod
    def create(cls, transform_name: str) -> DataTransform:
        key = transform_name.strip().lower()
        try:
            return cls._TRANSFORMS[key]()
        except KeyError as error:
            supported = ', '.join(sorted(cls._TRANSFORMS))
            raise ValueError(
                f"Unknown transformation '{transform_name}'. Supported names
            ) from error
```

In [27]:

```
# Use the fully cleaned DataFrame produced by Exercise 2 (FillMean strategy)
cleaned_input = cleaned_data.copy()
assert cleaned_input.isna().sum().sum() == 0

normalized_df = TransformFactory.create('normalize').apply(cleaned_input)
deduplicated_df = TransformFactory.create('remove_duplicates').apply(cleaned
standardized_df = TransformFactory.create('text').apply(cleaned_input)

print('Normalized numeric columns:')
display(normalized_df)

print('Duplicate rows removed:')
```

```
display(deduplicated_df)

print('Text columns standardized:')
display(standardized_df)

# Transformations can also be composed into a pipeline.
pipeline = ['text', 'normalize', 'remove_duplicates']
transformed_df = cleaned_input.copy()
for transform_name in pipeline:
    transformed_df = TransformFactory.create(transform_name).apply(transform

print('Composed pipeline result:')
display(transformed_df)
```

```
Normalized numeric columns:
```

	id	name	age	height_cm	weight_kg	city	score
0	0.000000	Alice	0.25000	0.333333	0.433333	New York	0.588235
1	0.166667	Bob	0.40625	0.800000	0.500000	Los Angeles	0.882353
2	0.333333	Charlie	1.00000	0.533333	0.566667	Chicago	0.647059
3	0.500000	David	0.40625	0.466667	0.833333	Houston	0.000000
4	0.666667	Eva	0.00000	0.000000	0.000000	New York	0.764706
5	0.833333	Frank	0.37500	1.000000	1.000000	Los Angeles	0.647059
6	1.000000	Grace	0.40625	0.133333	0.166667	Chicago	1.000000

```
Duplicate rows removed:
```

	id	name	age	height_cm	weight_kg	city	score
0	1	Alice	29.00	165.0	68.0	New York	85.0
1	2	Bob	30.25	172.0	70.0	Los Angeles	90.0
2	3	Charlie	35.00	168.0	72.0	Chicago	86.0
3	4	David	30.25	167.0	80.0	Houston	75.0
4	5	Eva	27.00	160.0	55.0	New York	88.0
5	6	Frank	30.00	175.0	85.0	Los Angeles	86.0
6	7	Grace	30.25	162.0	60.0	Chicago	92.0

```
Text columns standardized:
```

	id	name	age	height_cm	weight_kg	city	score
0	1	alice	29.00	165.0	68.0	new york	85.0
1	2	bob	30.25	172.0	70.0	los angeles	90.0
2	3	charlie	35.00	168.0	72.0	chicago	86.0
3	4	david	30.25	167.0	80.0	houston	75.0
4	5	eva	27.00	160.0	55.0	new york	88.0
5	6	frank	30.00	175.0	85.0	los angeles	86.0
6	7	grace	30.25	162.0	60.0	chicago	92.0

```
Composed pipeline result:
```

	id	name	age	height_cm	weight_kg	city	score
0	0.000000	alice	0.25000	0.333333	0.433333	new york	0.588235
1	0.166667	bob	0.40625	0.800000	0.500000	los angeles	0.882353
2	0.333333	charlie	1.00000	0.533333	0.566667	chicago	0.647059
3	0.500000	david	0.40625	0.466667	0.833333	houston	0.000000
4	0.666667	eva	0.00000	0.000000	0.000000	new york	0.764706
5	0.833333	frank	0.37500	1.000000	1.000000	los angeles	0.647059
6	1.000000	grace	0.40625	0.133333	0.166667	chicago	1.000000

Exercise 5: Build a Full Cleaning Pipeline (Template Method Pattern)

Objective: • Combine all previous concepts into one cleaning pipeline. • Use the Template Method Pattern to define a consistent workflow. Concept: The Template Method defines the skeleton of a process and lets subclasses override specific steps. Instructions:

1. Create an abstract class DataPipeline with a run() method defining steps: o load() o clean() o transform() o save()
2. Create CSVDataPipeline that implements each step using previous classes

In [28]:
```
#1      Create an abstract class DataPipeline with a run() method defining s
class DataPipeline(ABC):
    @log_action
    @log_time
    def run(self):
        """Execute the fixed load → clean → transform → save sequence."""
        self.load()
        self.clean()
```

```
        self.transform()
        self.save()
        return self

    @abstractmethod
    def load(self) -> None:
        ...

    @abstractmethod
    def clean(self) -> None:
        ...

    @abstractmethod
    def transform(self) -> None:
        ...

    @abstractmethod
    def save(self) -> None:
        ...
```

In [29]:
```
# 2.    Create CSVDataPipeline that implements each step using previous clas
class CSVDataPipeline(DataPipeline):
    def __init__(self, input_path, output_path, missing_strategy=None, trans
        self.input_path = input_path
        self.output_path = output_path
        self.cleaner = DataCleaner(missing_strategy)
        self.transforms = tuple(transforms) if transforms is not None else (
        self.df = None

    def load(self):
        self.df = CSVReader(self.input_path).data

    def clean(self):
        self.df = self.cleaner.clean(self.df)

    def transform(self):
        for name in self.transforms:
            transformer = TransformFactory.create(name)
            self.df = transformer.apply(self.df)

    def save(self):
        self.df.to_csv(self.output_path, index=False)
```

Create the sample dataset

The five input rows contain inconsistent headers and text casing, one duplicate Alice record, and three missing cells: age, salary, and department.

In [30]:
```
INPUT_PATH = Path("dirty_sample.csv")
OUTPUT_PATH = Path("cleaned_sample.csv")

raw_df = pd.DataFrame({
    " Full Name ": [" Alice ", "BOB", "Charlie ", " Alice ", "David"],
```

```
    " Age ": [25.0, None, 30.0, 25.0, 40.0],
    " Salary ": [50000.0, 60000.0, None, 50000.0, 75000.0],
    " Department ": [" HR ", "engineering", "FINANCE", " HR ", None],
})
raw_df.to_csv(INPUT_PATH, index=False)
```

Run the pipeline

Run the four steps and compare the original and cleaned data using standard pandas tables.

`FillMean` fills age with **30** and salary with **58,750** before deduplication. David's department remains missing because it is not numeric.

In [31]:
```
pipeline = CSVDataPipeline(
    input_path=INPUT_PATH,
    output_path=OUTPUT_PATH,
    missing_strategy=FillMean(),
    transforms=["normalize_columns", "standardize_text", "remove_duplicates"
).run()

cleaned_df = pipeline.df
print("Before cleaning:")
display(raw_df)
print("After cleaning:")
display(cleaned_df)
```

```
Starting run...
Starting clean...
clean took 0.000859 seconds.
Finished clean.
run took 0.003674 seconds.
Finished run.
Before cleaning:
```

	Full Name	Age	Salary	Department
0	Alice	25.0	50000.0	HR
1	BOB	NaN	60000.0	engineering
2	Charlie	30.0	NaN	FINANCE
3	Alice	25.0	50000.0	HR
4	David	40.0	75000.0	NaN

```
After cleaning:
```

	Full Name	Age	Salary	Department
0	alice	0.000000	0.00	hr
1	bob	0.333333	0.40	engineering
2	charlie	0.333333	0.35	finance
3	david	1.000000	1.00	NaN